

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 7

Local Data Storage

Menyimpan Data Lokal dengan DataStore dan SQLDelight

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026

OUTLINE PERTEMUAN

01

Review Minggu Lalu

Networking dan REST API

02

Pengenalan Local Storage

Jenis-jenis penyimpanan lokal

03

DataStore Preferences

Key-value storage untuk settings

04

SQLDelight

Type-safe SQL database untuk KMP

05

Offline-First Architecture

Caching dan sync strategies

06

Hands-on Practice

Implementasi Notes App dengan database

CAPAIAN PEMBELAJARAN

CPMK0501 dan CPMK0502

Mahasiswa mampu menjelaskan dan menerapkan konsep pemrograman untuk pengembangan perangkat lunak.

Indikator: Mampu menyimpan dan mengelola data lokal dengan DataStore dan SQLDelight

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Memahami jenis-jenis local storage di mobile
- ✓ Menggunakan DataStore untuk preferences/settings
- ✓ Menggunakan SQLDelight untuk database operations
- ✓ Menerapkan Repository pattern dengan local storage
- ✓ Mengimplementasikan offline-first architecture

REVIEW MINGGU LALU

Pertemuan 6: Networking dan REST API

HTTP/REST

GET, POST, PUT, DELETE methods

Ktor Client

HTTP client untuk Kotlin Multiplatform

Serialization

@Serializable untuk JSON parsing

Repository Pattern

Abstraksi data layer

UI States

Loading, Success, Error handling

? Pertanyaan: Bagaimana jika tidak ada koneksi internet? Bagaimana menyimpan data?

BAGIAN 1

Pengenalan Local Storage

Mengapa dan kapan menyimpan data lokal

MENGAPA LOCAL STORAGE?

Local storage memungkinkan aplikasi menyimpan data di device pengguna. Data tetap tersedia meskipun aplikasi ditutup atau tidak ada koneksi internet.

Keuntungan Local Storage:

 **Faster Access**

Data tersedia instan tanpa network latency

 **Offline Support**

Aplikasi tetap berfungsi tanpa internet

 **Data Persistence**

Data tetap ada setelah app ditutup

 **Battery Efficient**

Tidak perlu radio untuk network requests

 **Reduce Data Usage**

Hemat kuota dengan caching

JENIS-JENIS LOCAL STORAGE

DataStore

Use Case: Preferences, Settings

Example: Theme, Language, Token

SQLDelight

Use Case: Structured Data

Example: Notes, Users, Products

File Storage

Use Case: Binary Data

Example: Images, Documents, Cache

Secure Storage

Use Case: Sensitive Data

Example: Passwords, API Keys

Hari ini fokus: DataStore dan SQLDelight

BAGIAN 2

DATASTORE PREFERENCES

Key-Value Storage untuk Settings

APA ITU DATASTORE?

DataStore adalah solusi modern untuk menyimpan data key-value. Menggantikan SharedPreferences dengan API yang lebih aman menggunakan Kotlin Coroutines dan Flow.

DataStore vs SharedPreferences:

✗ SharedPreferences

- Synchronous API (blocking)
- No type safety
- No error handling
- Can cause UI jank
- No multiplatform support

✓ DataStore

- Asynchronous with Coroutines
- Type-safe with Proto/Preferences
- Built-in error handling
- Main thread safe
- KMP support (with libraries)

SETUP DATASTORE (KMP)

build.gradle.kts

```
// Menggunakan library multiplatform-settings untuk KMP
// atau androidx.datastore untuk Android-only

kotlin {
    sourceSets {
        commonMain.dependencies {
            // Multiplatform Settings (KMP compatible)
            implementation("com.russhwolf:multiplatform-settings:1.1.1")
            implementation("com.russhwolf:multiplatform-settings-coroutines:1.1.1")

            // Atau untuk Android-only project:
            // implementation("androidx.datastore:datastore-preferences:1.0.0")
        }
    }
}

// Untuk iOS, settings disimpan di UserDefaults
// Untuk Android, settings disimpan di SharedPreferences
```

SETTINGS MANAGER

SettingsManager.kt

```
import com.russhwolf.settings.Settings
import com.russhwolf.settings.get
import com.russhwolf.settings.set

class SettingsManager(private val settings: Settings) {

    companion object {
        private const val KEY_THEME = "app_theme"
        private const val KEY_LANGUAGE = "app_language"
        private const val KEY_ONBOARDING = "onboarding_complete"
        private const val KEY_AUTH_TOKEN = "auth_token"
    }

    // Theme: "light", "dark", "system"
    var theme: String
        get() = settings[KEY_THEME, "system"]
        set(value) { settings[KEY_THEME] = value }

    var language: String
        get() = settings[KEY_LANGUAGE, "en"]
        set(value) { settings[KEY_LANGUAGE] = value }

    var isOnboardingComplete: Boolean
        get() = settings[KEY_ONBOARDING, false]
        set(value) { settings[KEY_ONBOARDING] = value }

    var authToken: String?
        get() = settings.getStringOrNull(KEY_AUTH_TOKEN)
        set(value) {
            if (value != null) settings[KEY_AUTH_TOKEN] = value
            else settings.remove(KEY_AUTH_TOKEN)
        }
}
```

SETTINGS DENGAN FLOW

FlowSettingsManager.kt

```
import com.russhwolf.settings.coroutines.FlowSettings
import com.russhwolf.settings.coroutines.toFlowSettings
import kotlinx.coroutines.flow.Flow

class FlowSettingsManager(settings: Settings) {
    private val flowSettings: FlowSettings = settings.toFlowSettings()

    // Observe theme changes as Flow
    val themeFlow: Flow<String> = flowSettings.getStringFlow(KEY_THEME, "system")

    val languageFlow: Flow<String> = flowSettings.getStringFlow(KEY_LANGUAGE, "en")

    val isOnboardingCompleteFlow: Flow<Boolean> =
        flowSettings.getBooleanFlow(KEY_ONBOARDING, false)

    // Update settings
    suspend fun setTheme(theme: String) {
        flowSettings.putString(KEY_THEME, theme)
    }

    suspend fun setLanguage(language: String) {
        flowSettings.putString(KEY_LANGUAGE, language)
    }

    suspend fun completeOnboarding() {
        flowSettings.putBoolean(KEY_ONBOARDING, true)
    }
}
```

MENGGUNAKAN DI VIEWMODEL

SettingsViewModel.kt

```
class SettingsViewModel(  
    private val settingsManager: FlowSettingsManager  
) : ViewModel() {  
  
    val currentTheme: StateFlow<String> = settingsManager.themeFlow  
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), "system")  
  
    val currentLanguage: StateFlow<String> = settingsManager.languageFlow  
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), "en")  
  
    fun changeTheme(theme: String) {  
        viewModelScope.launch {  
            settingsManager.setTheme(theme)  
        }  
    }  
  
    fun changeLanguage(language: String) {  
        viewModelScope.launch {  
            settingsManager.setLanguage(language)  
        }  
    }  
}  
  
// Di Composable  
@Composable  
fun SettingsScreen(viewModel: SettingsViewModel) {  
    val theme by viewModel.currentTheme.collectAsState()  
    // Use theme value...  
}
```

BAGIAN 3

SQLDELIGHT

Type-Safe SQL Database untuk KMP

APA ITU SQLDELIGHT?

SQLDelight adalah library dari Square/CashApp yang generates type-safe Kotlin APIs dari SQL statements. Mendukung Kotlin Multiplatform dengan SQLite di semua platform.

Keuntungan SQLDelight:

- | | |
|-------------------------|---|
| ✓ Type-Safe | Compile-time verification untuk SQL queries |
| ✓ SQL First | Write real SQL, bukan DSL baru |
| ✓ Multiplatform | Support Android, iOS, Desktop, Web |
| ✓ Auto-Generated | Kotlin code generated dari .sq files |
| ✓ Reactive | Built-in support untuk Flow |

SETUP SQLDELIGHT

build.gradle.kts

```
plugins {
    id("app.cash.sqldelight") version "2.0.1"
}

kotlin {
    sourceSets {
        commonMain.dependencies {
            implementation("app.cash.sqldelight:runtime:2.0.1")
            implementation("app.cash.sqldelight:coroutines-extensions:2.0.1")
        }
        androidMain.dependencies {
            implementation("app.cash.sqldelight:android-driver:2.0.1")
        }
        iosMain.dependencies {
            implementation("app.cash.sqldelight:native-driver:2.0.1")
        }
    }
}

sqldelight {
    databases {
        create("NotesDatabase") {
            packageName.set("com.example.notes.db")
        }
    }
}
```

MENULIS SQL SCHEMA

File: src/commonMain/sqlDelight/com/example/notes/db/Note.sq

Note.sq

```
-- Create table
CREATE TABLE Note (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  updated_at INTEGER NOT NULL
);

-- Select all notes
selectAll:
SELECT * FROM Note ORDER BY updated_at DESC;

-- Select note by id
selectById:
SELECT * FROM Note WHERE id = ?;

-- Insert new note
insert:
INSERT INTO Note(title, content, created_at, updated_at)
VALUES (?, ?, ?, ?);

-- Update note
update:
UPDATE Note SET title = ?, content = ?, updated_at = ? WHERE id = ?;

-- Delete note
delete:
DELETE FROM Note WHERE id = ?;
```

DATABASE DRIVER (EXPECT/ACTUAL)

commonMain

```
// DatabaseDriverFactory.kt
expect class DatabaseDriverFactory {
    fun createDriver(): SqlDriver
}

// Di commonMain
class NotesDatabase(
    driverFactory: DatabaseDriverFactory
) {
    private val driver = driverFactory
        .createDriver()
    private val database = NotesDatabase(driver)
    val queries = database.noteQueries
}
```

androidMain

```
// DatabaseDriverFactory.android.kt
actual class DatabaseDriverFactory(
    private val context: Context
) {
    actual fun createDriver(): SqlDriver {
        return AndroidSqliteDriver(
            schema = NotesDatabase.Schema,
            context = context,
            name = "notes.db"
        )
    }
}
```

iosMain

```
// DatabaseDriverFactory.ios.kt
actual class DatabaseDriverFactory {
    actual fun createDriver(): SqlDriver {
        return NativeSqliteDriver(
            schema = NotesDatabase.Schema,
            name = "notes.db"
        )
    }
}
```

REPOSITORY DENGAN SQLDELIGHT

NoteRepository.kt

```
class NoteRepository(private val database: NotesDatabase) {

    private val queries = database.queries

    // Get all notes as Flow
    fun getAllNotes(): Flow<List<Note>> {
        return queries.selectAll()
            .asFlow()
            .mapToList(Dispatchers.IO)
    }

    // Get note by ID
    suspend fun getNoteById(id: Long): Note? {
        return withContext(Dispatchers.IO) {
            queries.selectById(id).executeAsOneOrNull()
        }
    }

    // Insert note
    suspend fun insertNote(title: String, content: String) {
        val now = Clock.System.now().toEpochMilliseconds()
        withContext(Dispatchers.IO) {
            queries.insert(title, content, now, now)
        }
    }

    // Update note
    suspend fun updateNote(id: Long, title: String, content: String) {
        val now = Clock.System.now().toEpochMilliseconds()
        withContext(Dispatchers.IO) {
            queries.update(title, content, now, id)
        }
    }

    // Delete note
    suspend fun deleteNote(id: Long) {
        withContext(Dispatchers.IO) {
```

VIEWMODEL DENGAN DATABASE

NotesViewModel.kt

```
class NotesViewModel(private val repository: NoteRepository) : ViewModel() {

    // All notes sebagai StateFlow
    val notes: StateFlow<List<Note>> = repository.getAllNotes()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    // Selected note untuk edit
    private val _selectedNote = MutableStateFlow<Note?>(null)
    val selectedNote: StateFlow<Note?> = _selectedNote.asStateFlow()

    fun addNote(title: String, content: String) {
        viewModelScope.launch {
            repository.insertNote(title, content)
        }
    }

    fun updateNote(id: Long, title: String, content: String) {
        viewModelScope.launch {
            repository.updateNote(id, title, content)
        }
    }

    fun deleteNote(id: Long) {
        viewModelScope.launch {
            repository.deleteNote(id)
        }
    }

    fun selectNote(id: Long) {
        viewModelScope.launch {
            _selectedNote.value = repository.getNoteById(id)
        }
    }
}
```

BAGIAN 4

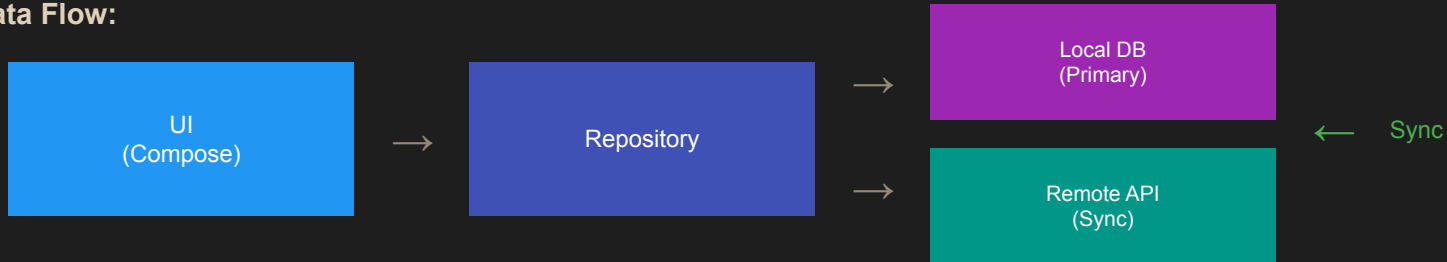
OFFLINE-FIRST ARCHITECTURE

Caching dan Sync Strategies

OFFLINE-FIRST ARCHITECTURE

Offline-first berarti aplikasi selalu menggunakan local database sebagai sumber utama data. Network hanya untuk sync. User experience tetap smooth meskipun tanpa koneksi.

Data Flow:



1. UI reads from Local DB (always available)
2. Repository syncs with Remote API in background
3. Changes pushed to Local DB → UI auto-updates via Flow

CACHING STRATEGIES

Cache First

Baca dari cache dulu, fetch jika expired

Use: Static content, user profiles

Network First

Fetch dari network, fallback ke cache

Use: News, real-time data

Stale While Revalidate

Return cache, update di background

Use: Feed, lists

Cache Only

Hanya dari cache, no network

Use: Offline-only features

REPOSITORY DENGAN CACHING

OfflineFirstRepository.kt

```
class NoteRepository(
    private val localDataSource: NoteLocalDataSource,
    private val remoteDataSource: NoteRemoteDataSource
) {
    fun getNotes(): Flow<List<Note>> {
        return localDataSource.getAllNotes()
            .onStart {
                // Sync dengan server di background
                refreshFromNetwork()
            }
    }

    private suspend fun refreshFromNetwork() {
        try {
            val remoteNotes = remoteDataSource.fetchNotes()
            localDataSource.insertAll(remoteNotes)
        } catch (e: Exception) {
            // Network error - gunakan data lokal
            // Log error tapi tidak throw ke UI
        }
    }

    suspend fun addNote(note: Note) {
        // Simpan ke lokal dulu (optimistic update)
        localDataSource.insert(note)

        // Sync ke server di background
        try {
            remoteDataSource.createNote(note)
        } catch (e: Exception) {
            // Mark as pending sync
            localDataSource.markPendingSync(note.id)
        }
    }
}
```

BAGIAN 5

HANDS-ON PRACTICE

3 Latihan Praktik

LATIHAN 1: SETTINGS DENGAN DATASTORE

Implementasi settings screen dengan preferences

Template

```
class SettingsManager(settings: Settings) {  
    var theme: String  
        get() = settings["theme", "system"]  
        set(value) { settings["theme"] = value }  
  
    var fontSize: Int  
        get() = settings["font_size", 16]  
        set(value) { settings["font_size"] = value }  
  
    var notificationsEnabled: Boolean  
        get() = settings["notifications", true]  
        set(value) { settings["notifications"] = value }  
}  
  
// TODO: Create SettingsScreen composable  
// TODO: Add theme selector (Light/Dark/System)  
// TODO: Add font size slider  
// TODO: Add notification toggle
```

Checklist:

- ☐ Setup multiplatform-settings
- ☐ Create SettingsManager
- ☐ Create SettingsViewModel
- ☐ Settings UI with options
- ☐ Persist changes
- ☐ Apply theme changes



Waktu: 20 menit

LATIHAN 2: NOTES APP DENGAN SQLDELIGHT

Implementasi CRUD operations dengan database

Note.sql

```
CREATE TABLE Note (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  content TEXT NOT NULL,  
  created_at INTEGER NOT NULL  
);  
  
selectAll:  
SELECT * FROM Note ORDER BY created_at DESC;  
  
insert:  
INSERT INTO Note(title, content, created_at)  
VALUES (?, ?, ?);  
  
update:  
UPDATE Note SET title = ?, content = ?  
WHERE id = ?;  
  
delete:  
DELETE FROM Note WHERE id = ?;  
  
-- TODO: Implement Repository  
-- TODO: Implement ViewModel  
-- TODO: Create Notes List Screen  
-- TODO: Create Add/Edit Note Screen
```

Checklist:

- ☐ Setup SQLDelight
- ☐ Write .sq schema file
- ☐ Create DatabaseDriver
- ☐ Implement Repository
- ☐ Create ViewModel
- ☐ Notes list with delete
- ☐ Add/Edit note form
- ☐ Flow for auto-update



Waktu: 30 menit

LATIHAN 3: OFFLINE-FIRST NEWS

Upgrade News Reader dengan caching

CachedNewsRepo.kt

```
class CachedNewsRepository(  
    private val api: NewsApi,  
    private val db: NewsDatabase  
) {  
    fun getNews(): Flow<List<Article>> {  
        return db.articleQueries.selectAll()  
            .asFlow()  
            .mapToList()  
            .onStart { refreshFromNetwork() }  
    }  
  
    private suspend fun refreshFromNetwork() {  
        try {  
            val articles = api.getTopHeadlines()  
            db.transaction {  
                db.articleQueries.deleteAll()  
                articles.forEach {  
                    db.articleQueries.insert(it)  
                }  
            }  
        } catch (e: Exception) {  
            // Use cached data  
        }  
    }  
}
```

Checklist:

- ☐ Article table schema
- ☐ Local data source
- ☐ Cached repository
- ☐ Stale-while-revalidate
- ☐ Show cached first
- ☐ Background refresh
- ☐ Offline indicator
- ☐ Pull to refresh



Waktu: 25 menit

BEST PRACTICES

✓ Single Source of Truth

Database sebagai satu-satunya sumber data untuk UI

✓ Reactive Updates

Gunakan Flow untuk auto-update UI saat data berubah

✓ Background Sync

Sync dengan server di background, jangan block UI

✓ Graceful Degradation

Aplikasi tetap berfungsi saat offline

✓ Conflict Resolution

Handle conflicts saat sync (last-write-wins, merge)

✓ Data Migration

Plan schema migrations untuk database updates

TROUBLESHOOTING

✗ SQLDelight: No database found

✓ Run Gradle sync, check .sq file location

✗ Type mismatch in query

✓ Pastikan types di Kotlin match dengan SQL schema

✗ Data not updating in UI

✓ Pastikan menggunakan Flow dan collectAsState()

✗ Database locked

✓ Use single database instance (singleton pattern)

RANGKUMAN PERTEMUAN 7

Local Storage

DataStore untuk settings, SQLDelight untuk structured data

DataStore

Key-value storage dengan Coroutines dan Flow

SQLDelight

Type-safe SQL dengan generated Kotlin code

Repository

Abstraksi data sources (local + remote)

Offline-First

Local DB sebagai primary, network untuk sync

Caching

Cache first, network first, stale-while-revalidate



Local Storage = Aplikasi yang reliable dan responsive!

TUGAS PRAKTIKUM MINGGU 7

Bobot: 4% | Deadline: Sebelum Pertemuan 8

Deskripsi Tugas:

Upgrade Notes App dengan fitur lengkap:

1. SQLDelight database untuk menyimpan notes
2. CRUD operations (Create, Read, Update, Delete)
3. Search functionality untuk mencari notes
4. Settings screen dengan DataStore (theme, sort order)
5. Offline-first: data tersimpan lokal
6. UI states yang proper (loading, empty, content)

Format Pengumpulan:

- Push ke GitHub repository (branch: week-7)
- README: Database schema, screenshot semua screens
- Video demo (45 detik) menunjukkan CRUD, search, settings, dan offline mode

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
SQLDelight Setup	20%	Schema, queries, driver setup
CRUD Operations	25%	All operations work correctly
DataStore Settings	15%	Preferences saved and applied
Search Feature	15%	Search works properly
UI/UX	15%	Clean UI, proper states
Code Quality	10%	Clean code, documentation
 Bonus (+10%) Sync dengan remote API  Penalti Terlambat: -10%/hari, Plagiat: 0		

SUMBER PUSTAKA

Dokumentasi

SQLDelight Official

cashapp.github.io/sqldelight

Dokumentasi

Multiplatform Settings

github.com/russhwolf/multiplatform-settings

Dokumentasi

Android DataStore

developer.android.com/topic/libraries/architecture/datastore

Guide

Offline-First Apps

developer.android.com/topic/architecture/data-layer/offline-first

Tutorial

SQLDelight KMP

kotlinlang.org/docs/multiplatform-mobile-ktor-sqldelight.html

Article

Repository Pattern

developer.android.com/topic/architecture/data-layer

PREVIEW MINGGU DEPAN

Pertemuan 8: Platform-Specific Code

expect/actual

Platform-specific implementations

Platform APIs

Camera, GPS, Sensors

Permissions

Runtime permission handling

Koin DI

Dependency Injection untuk KMP

Persiapan:

- Selesaikan tugas Notes App dengan database
- Baca dokumentasi expect/actual di KMP
- Pahami konsep Dependency Injection



SESI TANYA JAWAB

Ada pertanyaan tentang Local Data Storage?

CHECKLIST SEBELUM PULANG

Pastikan Anda sudah:

- ☐ Memahami perbedaan DataStore dan SQLDelight
- ☐ Bisa setup multiplatform-settings untuk preferences
- ☐ Bisa menulis SQL schema di .sq file
- ☐ Mengerti cara setup SQLDelight driver per platform
- ☐ Mampu implement Repository dengan database
- ☐ Memahami konsep offline-first architecture
- ☐ Mengerti berbagai caching strategies
- ☐ Mencatat deadline tugas Notes App

TIPS: DATABASE SCHEMA DESIGN

💡 Use INTEGER for timestamps

Simpan sebagai epoch milliseconds, lebih mudah query

💡 Always add created_at/updated_at

Berguna untuk sorting dan sync

💡 Use TEXT for IDs from server

UUID dari server biasanya String

💡 Add indexes for frequently queried columns

CREATE INDEX untuk performance

💡 Plan for migrations early

SQLDelight supports schema versioning

COMMON PATTERNS

Singleton Database

```
object DatabaseProvider {
    private var database: NotesDatabase? = null

    fun getDatabase(
        driverFactory: DatabaseDriverFactory
    ): NotesDatabase {
        return database ?: NotesDatabase(
            driverFactory.createDriver()
        ).also { database = it }
    }
}
```

Transaction

```
// Batch operations
suspend fun syncNotes(notes: List<Note>) {
    withContext(Dispatchers.IO) {
        database.transaction {
            queries.deleteAll()
            notes.forEach { note ->
                queries.insert(
                    note.title,
                    note.content,
                    note.createdAt
                )
            }
        }
    }
}
```

Flow with Mapping

```
// Map database entity to domain model
fun getAllNotes(): Flow<List<NoteUi>> {
    return queries.selectAll()
        .asFlow()
        .mapToList(Dispatchers.IO)
        .map { notes -> notes.map { it.toUiModel() } }
}

private fun Note.toUiModel() = NoteUi(
    id = id, title = title, content = content,
    formattedDate = Instant.fromEpochMilliseconds(created_at).toString()
)
```

Terima Kasih

Pertemuan 7: Local Data Storage

*"Data is the new oil.
Store it wisely, access it fast."*

- Mobile Development Wisdom

Selamat menyimpan data dengan aman! 🗄️